The planner will show the cost of building the bitmap with a Bitmap Heap Scan. A Bitmap Index Scan will show in the execution plan as a Scan Node when a Bitmap is used.

In this section, you were introduced to Bitmap Heap Scans and Bitmap Index Scans. When you see them in query execution plans, you'll now a little more about how PostgreSQL is trying to efficiently filter a set of rows. From there you may explore terminology in greater depth and whether an even more efficient method for filtering is possible given a specific query and specific table structure and row count.

Besides inefficient filtering operations, another type of performance problem is selecting too many rows. How can you find and address instances of that problem?

Adding Query Boundaries With Filtering and LIMIT

In Ruby on Rails applications operating at scale, you'll want to make sure queries in high frequency use cases filter to the smallest set of rows possible.

Try to add a `LIMIT` clause to most queries. Besides a LIMIT, try and add more boundaries to queries. Extra indexed fields helps the planner cut down the initial set of rows to scan.

In a multi-tenant application a `tenant_id` or `customer_id` column could be added to tables and indexed, so that a `WHERE` clause can be scoped just to those rows.

If your users are spread out among geographic regions or sales regions, consider grouping them into a regions table and scoping their records using an indexed `region_id` Foreign Key column.

Perhaps you can filter on Active and Inactive users.

Besides filtering in the `WHERE` clause, remember to add more conditions to `JOIN` operations.

For web applications, sometimes you'll need to have upper boundaries set and then introduce Pagination or Searching as user experience workarounds. You'll explore pagination techniques like Keyset Pagination later.

In the next section you'll move on to counting items efficiently.